

Bag of Tricks for Node Classification with Graph Neural Networks

Yangkun Wang¹, Jiarui Jin¹, Weinan Zhang¹, Yong Yu¹,
Zheng Zhang², David Wipf²

¹Shanghai Jiao Tong University

²Amazon

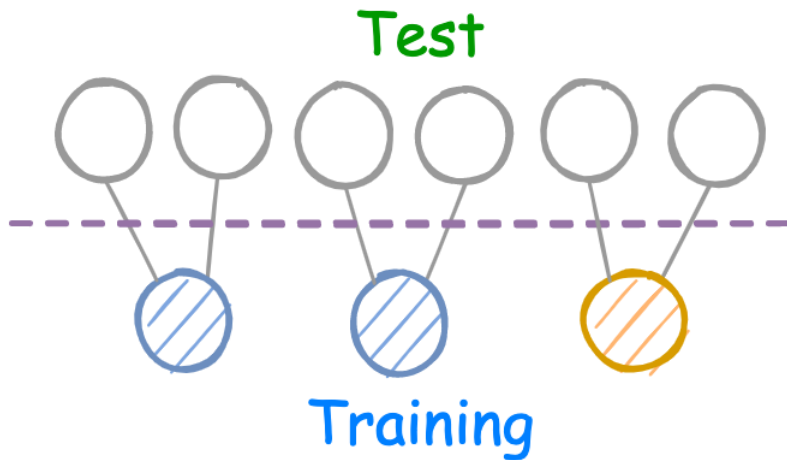
Labels in the Graph

Test

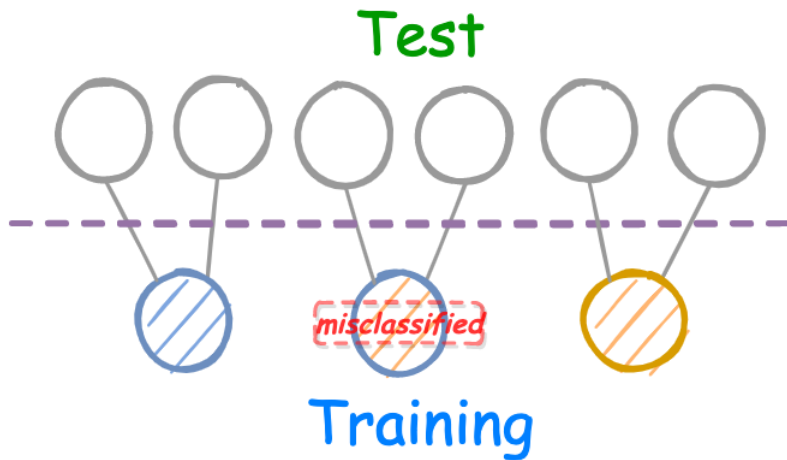


Training

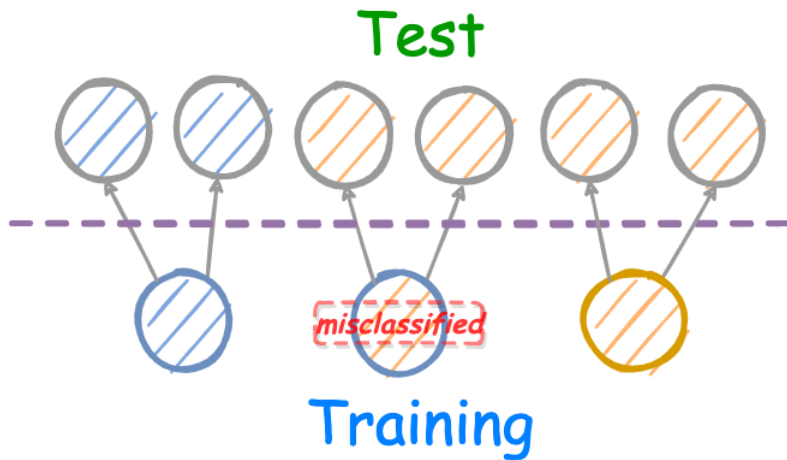
Labels in the Graph



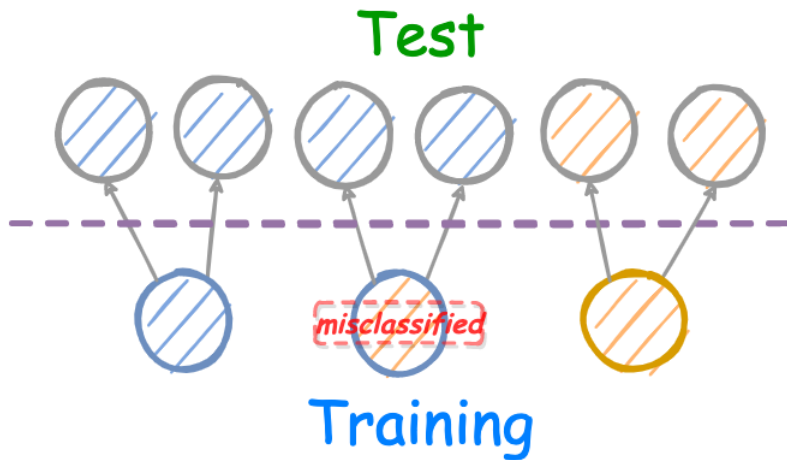
Labels in the Graph



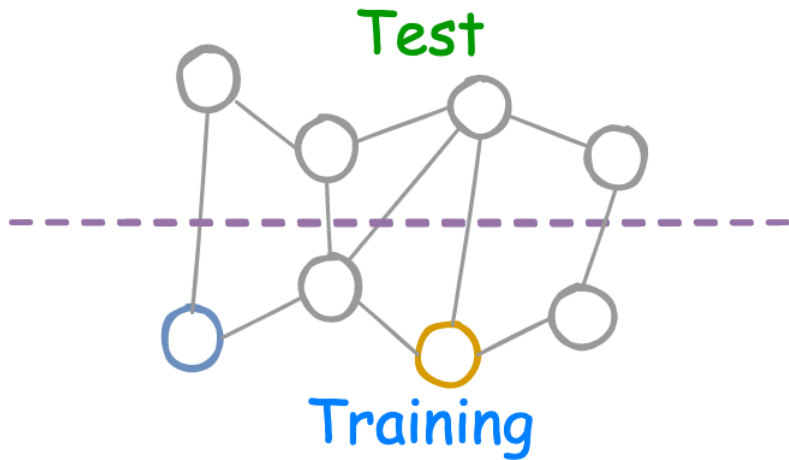
Labels in the Graph



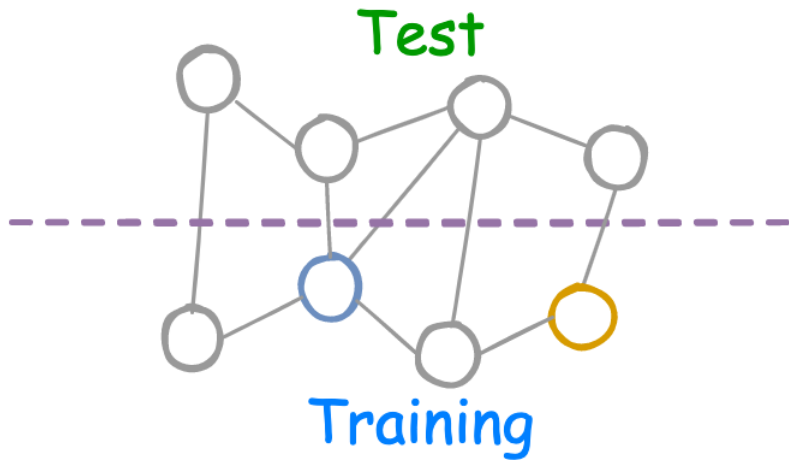
Labels in the Graph



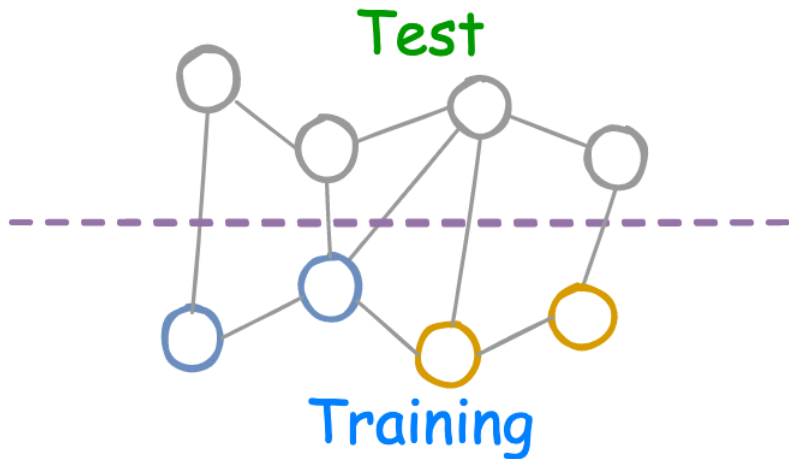
Splitting Strategy



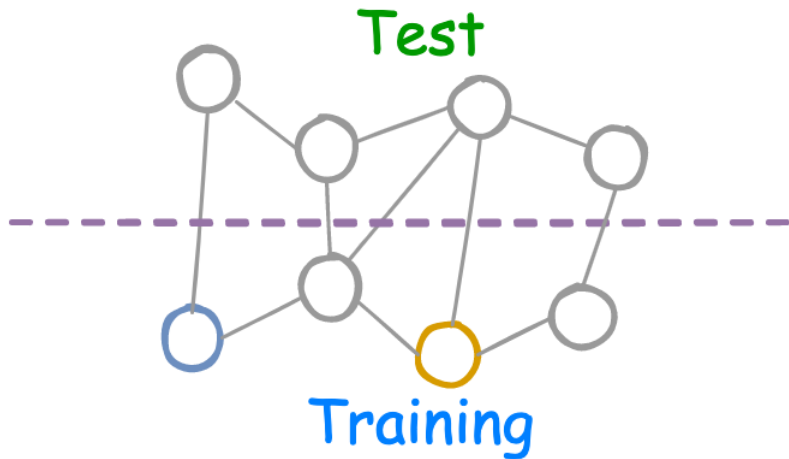
Splitting Strategy



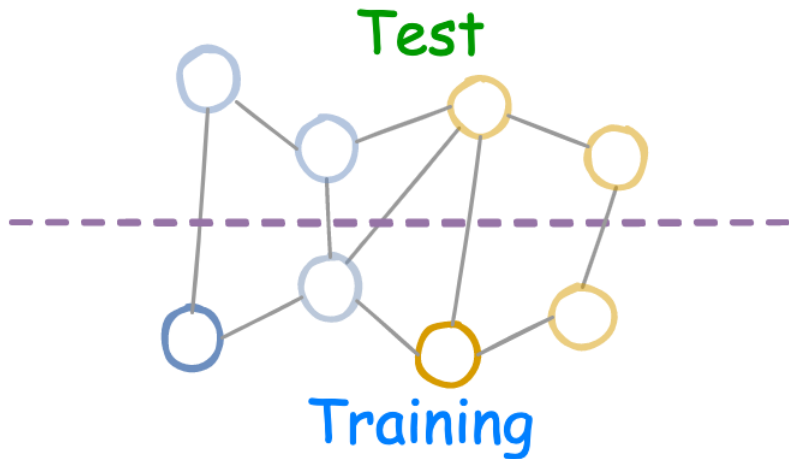
Splitting Strategy



Splitting Strategy



Splitting Strategy



Algorithm for Label Usage

Algorithm 1: Label Usage for Graph Neural Networks

INPUT: G, X, Y , the recycling times R

- 1: **for** each epoch **do**
 - 2: Obtain $\mathcal{D}_{train}^L, \mathcal{D}_{train}^U$ by randomly splitting $\mathcal{D}_{train}$
 - 3: $\mathbf{y}_i^L \leftarrow \begin{cases} \mathbf{y}_i, & (v_i, \mathbf{x}_i, \mathbf{y}_i) \in \mathcal{D}_{train}^L \\ 0, & \text{otherwise} \end{cases}$
 - 4: $\hat{\mathbf{Y}}^{(0)} \leftarrow f_{\theta}(X \parallel Y^L, A)$
 - 5: **for** $k \leftarrow 1$ to R **do**
 - 6: $\mathbf{y}_i^{(k-1)} \leftarrow \begin{cases} \mathbf{y}_i, & (v_i, \mathbf{x}_i, \mathbf{y}_i) \in \mathcal{D}_{train}^L \\ \hat{\mathbf{y}}_i^{(k-1)}, & \text{otherwise} \end{cases}$
 - 7: $\hat{\mathbf{Y}}^{(k)} \leftarrow f_{\theta}(X \parallel Y^{(k-1)}, A)$
 - 8: **end for**
 - 9: Compute $\mathcal{L}(\theta)$ and update θ via back propagation.
 - 10: **end for**
-

Algorithm for Label Usage

Algorithm 1: Label Usage for Graph Neural Networks

INPUT: G, X, Y , the recycling times R

```
1: for each epoch do
2:   Obtain  $\mathcal{D}_{train}^L, \mathcal{D}_{train}^U$  by randomly splitting  $\mathcal{D}_{train}$ 
3:    $\mathbf{y}_i^L \leftarrow \begin{cases} \mathbf{y}_i, & (v_i, \mathbf{x}_i, \mathbf{y}_i) \in \mathcal{D}_{train}^L \\ 0, & \text{otherwise} \end{cases}$ 
4:    $\hat{\mathbf{Y}}^{(0)} \leftarrow f_{\theta}(X \parallel Y^L, A)$ 
5:   for  $k \leftarrow 1$  to  $R$  do
6:      $\mathbf{y}_i^{(k-1)} \leftarrow \begin{cases} \mathbf{y}_i, & (v_i, \mathbf{x}_i, \mathbf{y}_i) \in \mathcal{D}_{train}^L \\ \hat{\mathbf{y}}_i^{(k-1)}, & \text{otherwise} \end{cases}$ 
7:      $\hat{\mathbf{Y}}^{(k)} \leftarrow f_{\theta}(X \parallel Y^{(k-1)}, A)$ 
8:   end for
9:   Compute  $\mathcal{L}(\theta)$  and update  $\theta$  via back propagation.
10: end for
```

- Propagate features and labels during both training and inference stages

Algorithm for Label Usage

Algorithm 1: Label Usage for Graph Neural Networks

INPUT: G, X, Y , the recycling times R

```
1: for each epoch do
2:   Obtain  $\mathcal{D}_{train}^L, \mathcal{D}_{train}^U$  by randomly splitting  $\mathcal{D}_{train}$ 
3:    $\mathbf{y}_i^L \leftarrow \begin{cases} \mathbf{y}_i, & (v_i, \mathbf{x}_i, \mathbf{y}_i) \in \mathcal{D}_{train}^L \\ 0, & \text{otherwise} \end{cases}$ 
4:    $\hat{\mathbf{Y}}^{(0)} \leftarrow f_{\theta}(X \parallel Y^L, A)$ 
5:   for  $k \leftarrow 1$  to  $R$  do
6:      $\mathbf{y}_i^{(k-1)} \leftarrow \begin{cases} \mathbf{y}_i, & (v_i, \mathbf{x}_i, \mathbf{y}_i) \in \mathcal{D}_{train}^L \\ \hat{\mathbf{y}}_i^{(k-1)}, & \text{otherwise} \end{cases}$ 
7:      $\hat{\mathbf{Y}}^{(k)} \leftarrow f_{\theta}(X \parallel Y^{(k-1)}, A)$ 
8:   end for
9:   Compute  $\mathcal{L}(\theta)$  and update  $\theta$  via back propagation.
10: end for
```

- Propagate features and labels during both training and inference stages
- Does not explicitly rely on the smooth assumption of label propagation

Algorithm for Label Usage

Algorithm 1: Label Usage for Graph Neural Networks

INPUT: G, X, Y , the recycling times R

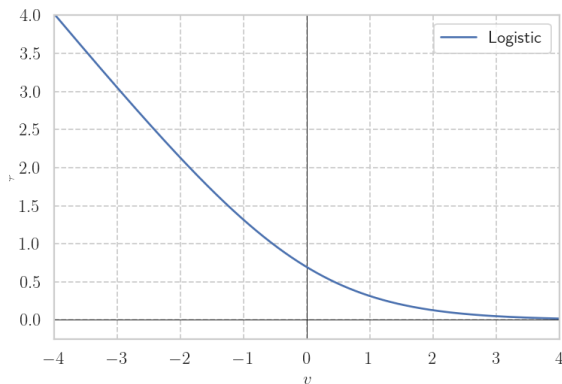
```
1: for each epoch do
2:   Obtain  $\mathcal{D}_{train}^L, \mathcal{D}_{train}^U$  by randomly splitting  $\mathcal{D}_{train}$ 
3:    $\mathbf{y}_i^L \leftarrow \begin{cases} \mathbf{y}_i, & (v_i, \mathbf{x}_i, \mathbf{y}_i) \in \mathcal{D}_{train}^L \\ 0, & \text{otherwise} \end{cases}$ 
4:    $\hat{\mathbf{Y}}^{(0)} \leftarrow f_{\theta}(X \parallel Y^L, A)$ 
5:   for  $k \leftarrow 1$  to  $R$  do
6:      $\mathbf{y}_i^{(k-1)} \leftarrow \begin{cases} \mathbf{y}_i, & (v_i, \mathbf{x}_i, \mathbf{y}_i) \in \mathcal{D}_{train}^L \\ \hat{\mathbf{y}}_i^{(k-1)}, & \text{otherwise} \end{cases}$ 
7:      $\hat{\mathbf{Y}}^{(k)} \leftarrow f_{\theta}(X \parallel Y^{(k-1)}, A)$ 
8:   end for
9:   Compute  $\mathcal{L}(\theta)$  and update  $\theta$  via back propagation.
10: end for
```

- Propagate features and labels during both training and inference stages
- Does not explicitly rely on the smooth assumption of label propagation
- Can be trained end-to-end

Logistic loss

Logistic loss

$$\phi_{\text{logit}}(v) = \log(1 + \exp(-v)) \quad (1)$$



Robust Loss Function for Classification

We consider weakening the convexity condition, and thereby designing a quasi-convex loss to contribute robustness.

ρ – *logit* loss

$$\phi_{\rho-\text{logit}}(v) := \rho(\phi_{\text{logit}}(v)), \quad (2)$$

where $\rho : \mathbb{R}^+ \rightarrow \mathbb{R}^+$ is a non-decreasing function.

Robust Loss Function for Classification

We consider weakening the convexity condition, and thereby designing a quasi-convex loss to contribute robustness.

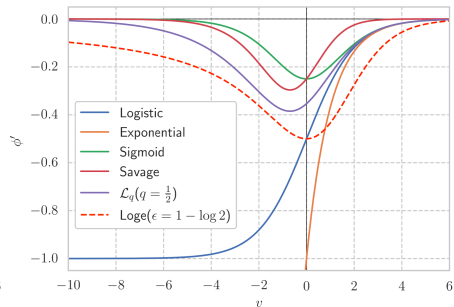
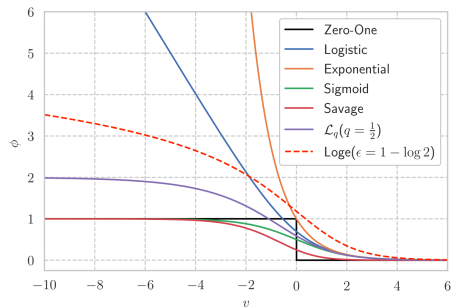
ρ – *logit* loss

$$\phi_{\rho-\text{logit}}(v) := \rho(\phi_{\text{logit}}(v)), \quad (2)$$

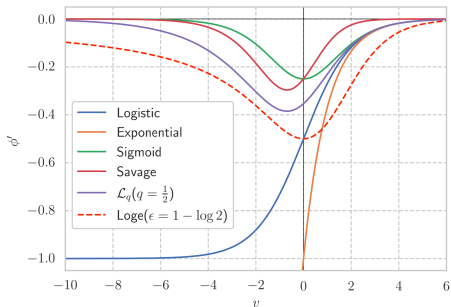
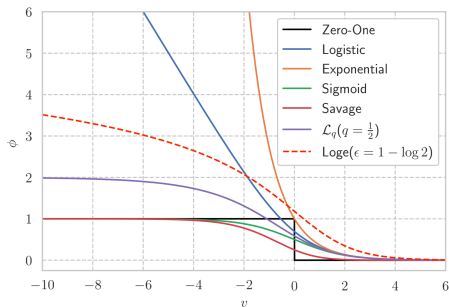
where $\rho : \mathbb{R}^+ \rightarrow \mathbb{R}^+$ is a non-decreasing function.

Loss	$\rho(z)$	$\rho(\phi_{\text{logit}}(v))$
Logistic	z	$\log(1 + \exp(-v))$
Exponential	$\exp(z) - 1$	$\exp(-v)$
Sigmoid	$1 - \exp(-z)$	$\frac{1}{1 + \exp(v)}$
Savage	$(1 - \exp(-z))^2$	$\frac{1}{(1 + \exp(v))^2}$
$\mathcal{L}_q$	$\frac{1}{q}(1 - \exp(-qz))$	$\frac{1}{q} \left(1 - \frac{1}{(1 + \exp(-v))^q} \right)$
Loge	$\log(\epsilon + z) - \log \epsilon$	$\log(\epsilon + \log(1 + \exp(-v))) - \log \epsilon$

Robust Loss Function for Classification

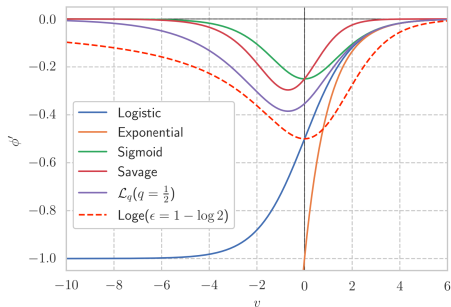
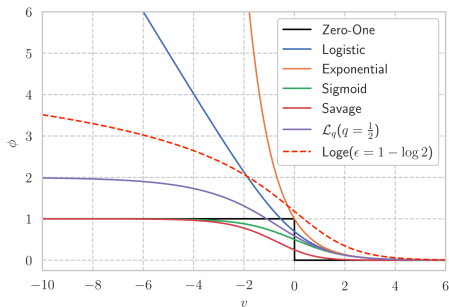


Robust Loss Function for Classification



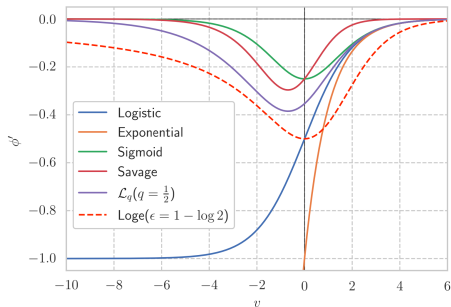
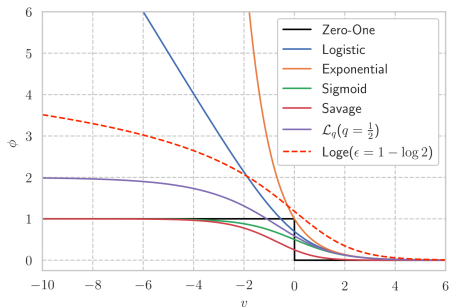
- Unbounded, and the derivative converges to 0 slowly as v decreases

Robust Loss Function for Classification



- Unbounded, and the derivative converges to 0 slowly as v decreases
- Enhance the gradient signal near the classification decision boundary

Robust Loss Function for Classification



- Unbounded, and the derivative converges to 0 slowly as v decreases
- Enhance the gradient signal near the classification decision boundary
- For $v > 0$, the derivative converges to 0 relatively quickly

Tweaking the GAT Architecture

GAT with symmetric normalized adjacency matrix

Let $\mathbf{A}_{att} = \mathbf{D}\alpha$, with α being the attention matrix,

$$\mathbf{x}^{(l+1)} = \sigma \left(\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}}_{att} \tilde{\mathbf{D}}^{-\frac{1}{2}} \mathbf{x}^{(l)} \mathbf{w}_0^{(l)} + \mathbf{x}^{(l)} \mathbf{w}_1^{(l)} \right), \quad (3)$$

where $\tilde{\mathbf{A}}_{att} = \mathbf{I} + \mathbf{A}_{att}$.

Tweaking the GAT Architecture

GAT with symmetric normalized adjacency matrix

Let $\mathbf{A}_{att} = \mathbf{D}\alpha$, with α being the attention matrix,

$$\mathbf{x}^{(l+1)} = \sigma \left(\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}}_{att} \tilde{\mathbf{D}}^{-\frac{1}{2}} \mathbf{x}^{(l)} \mathbf{w}_0^{(l)} + \mathbf{x}^{(l)} \mathbf{w}_1^{(l)} \right), \quad (3)$$

where $\tilde{\mathbf{A}}_{att} = \mathbf{I} + \mathbf{A}_{att}$.

Non-interactive attention

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T \mathbf{x}_j))}{\sum_{r \in \mathcal{N}(v_i)} \exp(\text{LeakyReLU}(\mathbf{a}^T \mathbf{x}_r))} \quad (4)$$

Tweaking the GAT Architecture

GAT with symmetric normalized adjacency matrix

Let $\mathbf{A}_{att} = \mathbf{D}\alpha$, with α being the attention matrix,

$$\mathbf{x}^{(l+1)} = \sigma \left(\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}}_{att} \tilde{\mathbf{D}}^{-\frac{1}{2}} \mathbf{x}^{(l)} \mathbf{w}_0^{(l)} + \mathbf{x}^{(l)} \mathbf{w}_1^{(l)} \right), \quad (3)$$

where $\tilde{\mathbf{A}}_{att} = \mathbf{I} + \mathbf{A}_{att}$.

Non-interactive attention

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T \mathbf{x}_j))}{\sum_{r \in \mathcal{N}(v_i)} \exp(\text{LeakyReLU}(\mathbf{a}^T \mathbf{x}_r))} \quad (4)$$

Attention involving edge features

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T [\mathbf{x}_i^V \parallel \mathbf{x}_j^V \parallel \mathbf{x}_{ij}^E]))}{\sum_{r \in \mathcal{N}(v_i)} \exp(\text{LeakyReLU}(\mathbf{a}^T [\mathbf{x}_i^V \parallel \mathbf{x}_r^V \parallel \mathbf{x}_{ij}^E]))} \quad (5)$$

Experiments

Dataset	Model	Label Usage	Accuracy(%)
Arxiv	GCN	–	72.48 ± 0.11
Arxiv	GCN	label as input	72.64 ± 0.10
Arxiv	GCN	label reuse	72.78 ± 0.17
Arxiv	GCN+linear	–	72.74 ± 0.13
Arxiv	GCN+linear	label as input	73.13 ± 0.14
Arxiv	GCN+linear	label reuse	73.22 ± 0.13
Arxiv	GAT	–	73.20 ± 0.16
Arxiv	GAT	label as input	73.24 ± 0.10
Arxiv	GAT	label reuse	73.43 ± 0.13
Arxiv	GAT(norm.adj.)	–	73.59 ± 0.14
Arxiv	GAT(norm.adj.)	label as input	73.66 ± 0.11
Arxiv	GAT(norm.adj.)	label reuse	73.91 ± 0.12
Arxiv	GAT(norm.adj.)	label reuse+C&S	73.95 ± 0.12
Arxiv	AGDN	–	73.75 ± 0.21
Arxiv	AGDN	label as input	73.98 ± 0.09
Proteins	GCN	–	80.07 ± 0.95
Proteins	GCN	label as input	80.80 ± 0.56
Proteins	GAT*	–	87.47 ± 0.16
Proteins	GAT*	label as input	87.65 ± 0.08

Dataset	Model	Accuracy(%)		
		Logistic	Savage	Loge
Cora	MLP	59.72 ± 1.01	61.10 ± 0.91	60.39 ± 0.74
Cora	GCN	82.26 ± 0.84	81.65 ± 0.74	82.60 ± 0.83
Citeseer	MLP	57.75 ± 1.05	59.60 ± 0.92	59.07 ± 0.98
Citeseer	GCN	71.13 ± 1.12	71.10 ± 1.22	72.49 ± 1.12
Pubmed	MLP	73.15 ± 0.68	73.39 ± 0.62	72.93 ± 0.65
Pubmed	GCN	78.89 ± 0.71	78.91 ± 0.63	78.93 ± 0.69
Reddit	MLP	72.98 ± 0.09	68.64 ± 0.29	73.12 ± 0.09
Reddit	GCN	95.22 ± 0.04	92.29 ± 0.48	95.18 ± 0.03
Arxiv	MLP	56.18 ± 0.14	51.97 ± 0.20	56.72 ± 0.15
Arxiv	GCN	71.77 ± 0.34	68.47 ± 0.32	72.43 ± 0.16
Arxiv	GAT	73.08 ± 0.26	69.58 ± 1.00	73.20 ± 0.16
Arxiv	GAT(norm.adj.)	73.29 ± 0.17	69.22 ± 1.48	73.59 ± 0.14
Products	MLP	62.90 ± 0.16	58.13 ± 1.03	63.20 ± 0.13
Products	GAT	80.99 ± 0.16	77.48 ± 0.14	81.39 ± 0.14

Dataset	Accuracy(%)	
	vanilla GAT	GAT+norm.adj.
Cora	83.41 ± 0.74	83.72 ± 0.74
Citeseer	71.92 ± 0.92	72.25 ± 1.04
Pubmed	78.43 ± 0.64	78.77 ± 0.54
Reddit	96.97 ± 0.04	97.06 ± 0.05
Arxiv	73.20 ± 0.16	73.59 ± 0.14

Experiments

Rank	Method	Test Accuracy	Validation Accuracy	Contact	References	#Params	Hardware	Date
1	AGDN+BoT+self-KD+C&S	0.7431 $\pm$ 0.0014	0.7518 $\pm$ 0.0009	Chuxiong Sun	Paper , Code	1,513,294	Tesla V100 (16GB GPU)	Jul 22, 2021
2	AGDN+BoT+self-KD	0.7428 $\pm$ 0.0017	0.7526 $\pm$ 0.0001	Chuxiong Sun	Paper , Code	1,513,294	Tesla V100 (16GB GPU)	Jul 22, 2021
3	RevGAT+N.Adj+LabelReuse+SelfKD	0.7426 $\pm$ 0.0017	0.7497 $\pm$ 0.0008	Guohao Li - DeepGCNs.org	Paper , Code	2,098,256	NVIDIA Tesla V100 (32GB GPU)	Jun 21, 2021
4	GAT-node2vec + BoT + self-KD	0.7420 $\pm$ 0.0004	0.7482 $\pm$ 0.0015	Huixuan Chi	Paper , Code	1,700,432	Tesla V100 (32GB)	Jun 28, 2021
5	GAT+label reuse+self KD	0.7416 $\pm$ 0.0008	0.7514 $\pm$ 0.0004	Shunli Ren(CMIC@SJTU)	Paper , Code	1,441,580	GeForce RTX 1080Ti (11GB GPU)	Dec 15, 2020
6	AGDN+BoT	0.7410 $\pm$ 0.0015	0.7522 $\pm$ 0.0007	Chuxiong Sun	Paper , Code	1,513,294	Tesla V100 (16GB GPU)	Jul 22, 2021
7	GAT-node2vec + BoT	0.7405 $\pm$ 0.0004	0.7482 $\pm$ 0.0015	Huixuan Chi	Paper , Code	1,700,432	Tesla V100 (32GB)	Jun 28, 2021
8	RevGAT+NormAdj+LabelReuse	0.7402 $\pm$ 0.0018	0.7501 $\pm$ 0.0010	Guohao Li - DeepGCNs.org	Paper , Code	2,098,256	NVIDIA Tesla V100 (32GB GPU)	Jun 21, 2021
9	GAT+label+reuse+topo loss	0.7399 $\pm$ 0.0012	0.7513 $\pm$ 0.0009	Mengyang Niu (DAMO DI)	Paper , Code	1,441,580	Tesla V100 (16GB)	Dec 10, 2020
10	AGDN (GAT-HA+3_heads+labels)	0.7398 $\pm$ 0.0009	0.7519 $\pm$ 0.0009	Chuxiong Sun	Paper , Code	1,508,555	Tesla V100 (32GB GPU)	Jan 3, 2021

- Hamed Masnadi-shirazi and Nuno Vasconcelos. 2009. On the Design of Loss Functions for Classification: theory, robustness to outliers, and SavageBoost. In NeurIPS.
- Zhilu Zhang and Mert R Sabuncu. 2018. Generalized cross entropy loss for training deep neural networks with noisy labels. In arXiv:1805.07836.
- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. In arXiv:1710.10903.
- Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: Datasets for machine learning on graphs. In arXiv:2005.00687.

Thank you

Full Paper Link: <https://arxiv.org/abs/2103.13355>

Thank you

Full Paper Link: <https://arxiv.org/abs/2103.13355>

Thanks for listening!